Lecture II - Control Structures for Your Code

Programming with Python

Dr. Tobias Vlćek



EP 2: The Curfew

A decree from the city

Overnight the city banned delivery after **22:00**. Your app can no longer just say yes to every order. Today the code starts making decisions, repeating work over every order, and tidying up Tobi's menu.

(Tobi's fix: "we just deliver yesterday's orders the next morning." A lawyer disagreed.)

Warm-up

Question 1

Three questions from Episode 1. Commit. Hands up **before** the reveal.

```
price = 8.90
print(f"{price * 2:.2f}")
```

a) `17.8` b) `17.80` c) `Error`

Answer 1

```
price = 8.90
print(f"{price * 2:.2f}")
```

```
17.80
```

b): `:.2f` always prints two decimals. That's the whole receipt trick.

Question 2

```
print(type("9.99"))
```

a) `float` b) `int` c) `str`

Answer 2

```
print(type("9.99"))
```

```
<class 'str'>
```

c): quotes make it text, no matter how numeric it looks. (Tobi learned this the hard way.)

Question 3

```
print(7 // 2, 7 % 2)
```

a) `3.5 1` b) `3 1` c) `3 0.5`

Answer 3

```
print(7 // 2, 7 % 2)
```

```
3 1
```

b): `//` floors, `%` gives the remainder. Together: "how many fit, what's left."

Making Decisions

Asking a yes/no question

A **comparison** asks a question and answers with a **boolean**: `True` or `False`, the `bool` type you met last week:

```
delivery_hour = 21
print(delivery_hour < 22)    # before curfew?
print(delivery_hour == 22)   # exactly at curfew?
```

```
True
False
```

The comparison operators

Six operators, all returning `True` or `False`:

- `<` less than · `>` greater than
- `<=` at most · `>=` at least
- `==` equal · `!=` not equal

> ⚠ Warning
> `=` **assigns** a value; `==` **compares** two values. Mixing them up is the classic first-week bug.

Combining conditions

`and`, `or`, `not` glue booleans together:

```
is_weekday = True
before_curfew = 21 < 22
print(is_weekday and before_curfew)   # both must be True
```

```
True
```

`and` needs **both** sides true; `or` needs **at least one**; `not` flips it.

Predict: Tobi's weekend rule

Tobi wants "not a weekday, and still before curfew" to mean *deliver*. It is a Tuesday, 23:00. What prints?

```
is_weekday = True
before_curfew = False
print(not is_weekday and before_curfew)
```

a) `True` b) `False` c) `Error`

Predict. Pick a letter, then I reveal the answer.

Answer: `not` binds first

```
is_weekday = True
before_curfew = False
print(not is_weekday and before_curfew)
```

```
False
```

b) `False`: `not` only flips the value right next to it, so Python reads `(not is_weekday) and before_curfew`, which is `False and False`. To flip the whole pair, put parentheses: `not (is_weekday and before_curfew)`.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_02_a/

Any questions? Let me know!

`if`: do something only when

An `if` runs its indented block **only** when the condition is `True`:

```
delivery_hour = 21
if delivery_hour < 22:
    print("Open: send the courier.")
```

```
Open: send the courier.
```

The **indentation** (4 spaces) is what marks the block. Python is strict about it.

`if` / `else`: two paths

`else` catches every case the `if` missed:

```
delivery_hour = 23
if delivery_hour < 22:
    print("Open: send the courier.")
else:
    print("Curfew: kitchen closed.")
```

```
Curfew: kitchen closed.
```

`if` / `elif` / `else`: a ladder

More than two cases? Stack `elif` branches. Reward loyal customers by tier:

```
past_orders = 8
if past_orders >= 20:
    tier = "Gold"
elif past_orders >= 5:
    tier = "Silver"
else:
    tier = "Bronze"
print(tier)
```

```
Silver
```

Predict: Tobi reorders the ladder

Tobi rewrote the tier ladder “to keep the common case on top”. A customer with **25 past orders** walks in. Which tier do they get?

```
past_orders = 25
if past_orders >= 5:
    tier = "Silver"
elif past_orders >= 20:
    tier = "Gold"
else:
    tier = "Bronze"
print(tier)
```

a) `Gold` b) `Bronze` c) `Silver`

Predict. Pick a letter, then I reveal the answer.

Answer: the first true branch wins

```
past_orders = 25
if past_orders >= 5:
    tier = "Silver"
elif past_orders >= 20:
    tier = "Gold"
else:
    tier = "Bronze"
print(tier)
```

```
Silver
```

c) `Silver`: Python checks branches top to bottom and takes the first `True` one. `25 >= 5` is already `True`, so the `Gold` branch is never even looked at. Order your ladder from strictest to loosest.

(One misplaced `elif` and Tobi demotes every VIP.)

Predict: exactly at curfew

The curfew is 22:00. An order comes in at **exactly** 22:00. What prints?

```
delivery_hour = 22
print(delivery_hour < 22)
```

a) `False` b) `True` c) `Error`

Predict. Pick a letter, then I reveal the answer.

Answer: `22 < 22`

```
delivery_hour = 22
print(delivery_hour < 22)
```

```
False
```

a) `False`: `<` is **strict**. 22 is not *less than* 22, so at 22:00 sharp the kitchen is already closed. Use `<=` when the boundary should count.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_02_b/



Any questions? Let me know!

Doing It Many Times

The copy-paste pain

Tobi totals the day's orders by hand, one `print` per order:

```
print("Falafel Wrap")
print("Pad Thai")
print("Miso Ramen")
```

```
Falafel Wrap
Pad Thai
Miso Ramen
```

Three lines for three items. A hundred orders? A hundred lines. There is a better way.

`for`: one step for every item

A **list** holds several values in square brackets. A **`for` loop** runs its block once for each item in it:

```
menu = ["Falafel Wrap", "Pad Thai", "Miso Ramen"]
for item in menu:
    print(item)
```

```
Falafel Wrap
Pad Thai
Miso Ramen
```

`item` is the **loop variable**. It takes each value in turn, and the name is yours to choose.

The accumulator pattern

Start a total at 0, then **add each item** as the loop visits it:

```
prices = [5.00, 3.50, 6.50]
total = 0
for price in prices:
    total = total + price
print(total)
```

```
15.0
```

After the loop, `total` holds the sum. This is how you replace Tobi's hand-counted dashboard.

`range()`: count without a list

`range(n)` counts from `0` up to, but **not including**, `n`:

```
for i in range(3):
    print(i)      # ping the courier's phone 3 times
```

```
0
1
2
```

Three passes: `0`, `1`, `2`. (It stops *before* 3, a beginner's favorite surprise.)

`range()` with a start and a step

- `range(start, stop)`: begin at `start`, stop before `stop`
- `range(start, stop, step)`: jump by `step` each time

```
print(list(range(2, 5)))       # list() shows the range as a list: 2, 3, 4
print(list(range(0, 10, 3)))   # every 3rd order gets a flyer: 0, 3, 6, 9
```

```
[2, 3, 4]
[0, 3, 6, 9]
```

Predict: pinging orders 1 to 5

Tobi pings the courier once per order, "for orders 1 to 5". How many pings?

```
pings = 0
for order in range(1, 5):
    pings = pings + 1
print(pings)
```

a) `4` b) `5` c) `1`

Predict. Pick a letter, then I reveal the answer.

Answer: `range(1, 5)` stops before 5

```
pings = 0
for order in range(1, 5):
    pings = pings + 1
print(pings)
```

```
4
```

a) `4`: `range(1, 5)` yields `1, 2, 3, 4`. The stop value is never included, no matter where you start. Order 5 never gets its ping.

Looping over a string

A string is a sequence too, and a `for` loop walks it character by character:

```
for letter in "Pad":
    print(letter)
```

```
P
a
d
```

A loop in one line (a preview)

A **list comprehension** is a `for` loop compressed into one line that builds a list:

```
doubled = [n * 2 for n in [4, 5, 6]]
print(doubled)
```

```
[8, 10, 12]
```

> **Note**
> Just a taste. Session IV gives comprehensions their proper treatment. For now: recognize the shape.

Predict: where does `print` run?

The `print` is **not** indented. What does this show?

```
total = 0
for n in [10, 20]:
    total = total + n
print(total)
```

a) `10` b) `10` then `30` c) `30`

Predict. Pick a letter, then I reveal the answer.

Answer: the unindented `print`

```
total = 0
for n in [10, 20]:
    total = total + n
print(total)
```

```
30
```

c) `30`: `print` sits outside the loop, so it runs once, after the loop finishes, showing the final total. Indent it and it would print on every pass. Indentation decides what repeats.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_02_c/



Any questions? Let me know!

While, and Clean Text

`while`: repeat as long as

A **`while` loop** repeats **as long as** its condition stays `True`. Suppose MunchCorp undercuts you. Cut your price each round until you drop below their price:

```
price = 10.00
rounds = 0
while price >= 7.00:
    price = round(price * 0.8, 2)   # 20% off each round
    rounds = rounds + 1
print(rounds, price)
```

```
2 6.4
```

Two rounds and you're under 7.00. (The lab's price war against MunchCorp is waiting after the break.)

The loop must move toward its goal

Every pass must nudge the condition **closer to `False`**: here the price shrinks, so the loop is guaranteed to end.

If nothing changes, the condition never flips and the loop runs **forever**:

```
# NEVER run this. It never stops, and freezes the browser tab
count = 1
```

```
while count > 0:
    count = count + 1    # count only grows, so the condition stays True
forever
```

> ⚠ **Warning**
> An endless `while` will freeze your marimo tab, and a frozen tab can only be closed. Check that **something inside the loop changes** the condition, and save and download before you experiment.

`break`: an early exit

`break` jumps out of a loop immediately, handy with `while True`:

```
count = 0
while True:
    count = count + 1
    if count == 3:
        break
print(count)
```

```
3
```

The loop would run forever, but `break` stops it the moment `count` hits 3.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_02_d/



Any questions? Let me know!

Cleaning up text

Tobi typed the daily special IN ALL CAPS with stray spaces. Strings have **methods** that return a cleaned-up **copy**:

```
raw = "  miso ramen  "
print(raw.strip())          # drop outer spaces
print(raw.strip().title())  # then Capitalise Each Word
print("special".upper())    # SHOUT
print("MOIN".lower())       # whisper
```

```
miso ramen
Miso Ramen
SPECIAL
moin
```

Methods can be **chained** left to right: `raw.strip().title()`.

Predict: did Tobi clean it?

Tobi calls `.strip()` on the special and prints it in brackets. What shows?

```
special = "  miso ramen  "
special.strip()
print("[" + special + "]")
```

a) `Error` b) `[miso ramen]` c) `[  miso ramen  ]`

Predict. Pick a letter, then I reveal the answer.

Answer: `.strip()` hands back a copy

```
special = "  miso ramen  "
special.strip()
print("[" + special + "]")
```

```
[  miso ramen  ]
```

c) **`[  miso ramen  ]`**: `.strip()` returns a **new** string and leaves `special` untouched; Tobi threw the clean copy away. Keep it by assigning: `special = special.strip()`.

Trimming specific characters

`.strip()` drops spaces; `.rstrip("!")` drops trailing `!`. Chain them to fix Tobi's shouting:

```
print("SALE!!!".rstrip("!"))           # SALE
print("SALE!!!".rstrip("!").title())   # Sale
```

```
SALE
Sale
```

The original string is never changed. Each method hands back a **new** one.

Predict: chained methods

What does this print?

```
print("  MOIN  ".strip().lower())
```

a) `MOIN` b) `moin` c) `Error`

Predict. Pick a letter, then I reveal the answer.

Answer: `"  MOIN  ".strip().lower()`

```
print("  MOIN  ".strip().lower())
```

```
moin
```

b) **`moin`**: `.strip()` hands back a new string, so `.lower()` has something to work on. Chained methods run left to right, each acting on the previous one's output.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_02_e/

Any questions? Let me know!

Checkpoint 0

A practice board review. Fifteen minutes, before the lab, **ungraded**. Same format as the five real checkpoints, so that nothing is new next week when it counts.

- **Individual work**: no AI, no neighbors, no messaging
- Slides, the cheatsheet and your own notebooks **may be open**
- The **link is on Moodle**. Open it and start
- 3 short tasks: compute a price, trace a line, fix one of Tobi's bugs
- Fill in your **name and student ID** as code at the top, exactly as you will next week

When you're done

Cmd/Ctrl+S, then menu → *Download* → *Download Python code* → upload the `.py` to the "Checkpoint 0" assignment on Moodle. Nothing is graded today. The point is that you have done the download and the upload once before it counts.

> **Note**
> Next week this exact routine is **Checkpoint 1**: 40 minutes, 12 points, Sessions I-II. If something goes wrong today, it costs nothing.

To the Lab

After the break: the lab

- Head to the lab notebook: Episode 2: The Curfew
- You'll enforce the 22:00 curfew, total the day's orders in a loop, de-shout Tobi's menu, and fight the **MunchCorp price war** with a `while` loop
- It runs entirely in your browser: no setup, just click and code

> ! **Important**
> **Download your `.py` before you leave.** Closing the tab without downloading loses your work, and downloading is exactly how you hand in the checkpoints.

Wrap-up

Three things to remember

1. `if` / `elif` / `else` run the first true branch: test the strictest condition first
2. `for` repeats over every item (accumulator: start at 0, add each); `while` repeats until its condition flips, and something inside must move it there
3. **String methods** (`.strip()`, `.title()`, `.upper()`, `.rstrip("!")`) return a cleaned copy, and chain left to right

> i **Note**
> **Next time, Episode 3** starts with Checkpoint 1: 40 minutes, no AI, everything from Episodes 1-2, so keep this notebook and the last one close. Then: Tobi has pasted the same receipt code 14 times, and one small change now takes him an afternoon. We bring in functions.

Literature

Books to start with

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Link to free online version
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

> i **Note**
> Nothing new here, but these are still great books to start with!

For more, see the literature list of this course.